RE4A User Guide

RE4A — Reverse Engineering for Automotive (SAD/SDD generator)

RE4A turns automotive C/C++ ECU source code into ASPICE-style design documents — a **Software Architecture Design (SAD)** and a **Software Detail Design (SDD)** — together with diagrams, a traceability matrix, an **Enterprise Architect** project, and **ALM** exports (ReqIF file or a direct REST push to Polarion / codeBeamer). It parses your code *without a compiler*, so it works even on source that does not build locally.

This guide is for **end users** of the RE4A desktop application (`RE4A.exe`) and command-line tool (`re4a` / `RE4A.Cli.exe`).

Table of contents

1. System requirements
2. Installation and first launch
3. Licensing and activation
4. Quick start
5. The main window
6. Working with projects
7. Scanning your source code
8. Reviewing scan results
9. Editing the SAD / SDD (preview editors)
10. Generating documents
11. Templates
12. AI assist (optional)
13. Enterprise Architect export
14. ALM export (ReqIF and REST push)
15. Settings reference
16. Command-line interface
17. Diagram rendering tools
18. Troubleshooting
19. Reference appendix

---

1. System requirements

Requirement	Needed for	Notes
Windows 64-bit	Everything	RE4A is a Windows desktop (WinForms) application.
.NET 10 runtime	Everything	Both `RE4A.exe` and `RE4A.Cli.exe` target .NET 10.
A physical network adapter (wired or Wi-Fi, enabled)	Licensing	Required to compute the machine fingerprint.

Requirement	Needed for	Notes
A valid RE4A license (`.lic`)	Every real operation	Per-machine, issued by your vendor. See §3.
Sparx Enterprise Architect 16 or 17 (64-bit)	EA (`.qea`) export only	Provides the blank template and opens the result. Not bundled with RE4A.
Java + PlantUML (`plantuml.jar`)	Rendering diagrams as images	*Optional.* Without it, diagrams appear as text. See §17.
Graphviz (`dot.exe`)	Higher-quality diagram layout	*Optional.* Falls back to the built-in *smetana* engine.

> **Pandoc is NOT required.** Word (`.docx`) output is produced by a built-in writer inside RE4A — no external converter is involved.

Your source code **does not need to compile**. RE4A uses a preprocessor-less parser; the *coding-standard profiles* and *macro rules* exist precisely because no real compiler runs.

---

2. Installation and first launch

1. Copy the RE4A delivery folder to your PC (it contains `RE4A.exe`, `RE4A.Cli.exe` and their supporting DLLs). No installer is required.
2. Double-click **`RE4A.exe`**.
3. On first launch the app is **locked** until a license is installed — the License dialog opens automatically and a red banner appears at the top of the window. Follow §3 to activate.

Once activated, the app stays unlocked on later runs (the license is cached on your machine).

---

3. Licensing and activation

RE4A is a commercial product protected by a **per-machine** license. Activation is **fully offline** — there is no online sign-in or phone-home. You send your vendor a **Machine ID** once, and import the signed **`.lic`** file they return.

Activate a license

1. Click **License…** on the coloured banner at the top of the window (also available at **Options ▸ License…**).
2. In the dialog, read **This machine's ID** — a four-group code such as `XXXX-XXXX-XXXX-XXXX`. Click **Copy** (to clipboard) or **Save…** (to a text file, default `re4a-machine-id.txt`) and send it to your vendor.
3. When you receive your license, click **Import license file (.lic)…**, pick the file, and confirm.
4. On success the dialog reports **"Licensed to *<name>*. Valid until *<date>*"** and the application unlocks.

The imported license is stored at `%LOCALAPPDATA%\RE4A\license.lic`.

Key facts

- **One license = one machine.** A license copied to another PC will not work.

- The **Machine ID** is derived from four hardware traits (Windows Machine GUID, CPU ID, motherboard serial, primary network-adapter MAC). It contains no personal data. Removable USB and virtual network adapters are ignored, so plugging/unplugging a dock does not break your license.
- **Renewal:** within 14 days of expiry an amber banner warns you. Ask your vendor for a renewed `.lic` and import it the same way. If the machine is unchanged you do **not** need to re-send your Machine ID.
- The **command-line tool uses the same license** — there is no separate CLI activation. Activate in the desktop app; the CLI reuses the installed license.

License states

If a license problem occurs, the banner shows the reason:

State	Meaning	What to do
Missing	No license installed	Import a `.lic`.
Invalid	Corrupt, or not issued by this vendor	Obtain a correctly-issued license; check the file is not truncated.
WrongMachine	Issued for a different PC	Request a `.lic` for this machine's ID.
Expired	Past its expiry date	Renew.
Revoked	Revoked by the vendor	Contact your vendor.
ClockTampered	System clock rolled back	Set the correct date/time; if it persists, re-import the `.lic`.
FingerprintError	Hardware identity unreadable	Enable a wired/Wi-Fi adapter and retry (not a license problem).

> The **RE4A License Manager** is a vendor-internal tool. End users never run it — you only read your Machine ID and import the `.lic` here.

---

4. Quick start

The fastest path from source folder to documents:

1. **File ▸ New project from folder...** (Ctrl+N) → choose your ECU source root. RE4A proposes a Core / Component / Sub-component tree.
2. Adjust the tree if needed (drag files/folders, rename, add/remove nodes) and fill in component descriptions in the **Properties** tab.
3. Pick a **Coding standard** (default *AUTOSAR Classic*) under the tree.
4. Click **Scan**. Review the **Log** and **Issues** tabs.
5. Go to the **Generate** page → **Generate documents...** → choose an output folder.

You now have `SAD.md`, `SDD.md`, diagrams, `traceability.csv/.md`, `model.xml` and `metrics.csv`. Optionally export to **Enterprise Architect** or **ALM** from the same Generate page.

The equivalent one-liner on the command line:

```
re4a all C:\src\MyEcu -o C:\out --docx
```

---

5. The main window

RE4A is organised as a four-step wizard. The **step bar** at the top switches between pages; the **Log** and **Issues** tabs stay docked at the bottom on every page.

```
[ Scan ] ▸ [ SDD Preview ] ▸ [ SAD Preview ] ▸ [ Generate ]
```

- **Scan** — the project tree, scan options and the **Scan** button (the startup page).
- **SDD Preview** / **SAD Preview** — Markdown editors with live rendered preview.
- **Generate** — buttons to generate documents and export to EA / ALM.

Menu bar

Menu	Items
File	New project from folder… (Ctrl+N) · Open project… (Ctrl+O) · Save project (Ctrl+S) · Save project as… (Ctrl+Shift+S) · Exit
Options	Settings… · AI assist… · License…
Help	About RE4A

Toolbar

Quick buttons: **New**, **Open**, **Save**, and (right-aligned) **Settings**. The *Coding standard* selector and the *Scan* button are **not** on the toolbar — they live on the Scan page under the tree.

> While a scan, generation or export runs, the whole page area and the step bar are disabled and a progress bar animates. If no usable license is present, the entire UI is locked except the banner and status bar.

---

6. Working with projects

A **project** is a single `*.re4a.json` file describing your ECU: which source files exist and how they are grouped, plus the free text you supply and the parsing options. Alongside it, RE4A keeps a sibling scan database (`*.db`) and per-project sidecar files for your edited SDD/SAD text.

Create, open, save

- **New: File ▸ New project from folder…** → pick the source root ("Select the root folder of the ECU source code"). RE4A auto-proposes the tree (one Core `Core0`; each top-level source folder becomes a Component; nested folders become Sub-components). Only `.c .h .cpp .hpp .cc .hh .cxx` files are picked up.

- **Open: File ▸ Open project…** → choose a `*.re4a.json`.
- **Save: Ctrl+S** / the **Save** button. The window title shows the project name with a trailing `*` when there are unsaved changes.

The project tree

The left panel of the Scan page shows the hierarchy:

```
ECU
└─ Core: <name>
   └─ [SWC] <component>
      └─ [SUB] <sub-component>
         └─ <file>.c
```

- **Select:** left-click. **Multi-select** siblings of the same kind with **Ctrl+click** or **Shift+click**.
- **Re-group: drag** a selection onto a valid parent. The rules are: files → sub-components, sub-components → components, components → cores. The whole multi-selection moves as a group; the tree keeps its expand/scroll state and selects the moved nodes afterward.
- **Right-click** for context actions (the menu changes by node type):

Node	Actions
ECU root	Add core
Core	Add component · Rename… · Delete core
Component	Add sub-component · Rename… · Delete component
Sub-component	Rename… · Delete sub-component
File	Remove file from project (or *Remove N files…* for a multi-selection)

> Deleting a core/component/sub-component cascades to everything under it and **cannot be undone** (it asks for confirmation). Tree edits mark the project dirty but do **not** change the scan results until you **re-scan**.

Properties tab (documentation fields)

Select a component or sub-component to edit the metadata that feeds the generated documents:

- **Component:** ID, Description, Defined goal, *3rd party* (checkbox) + name, *Safety* (checkbox — safety components get a `Safe` prefix in generated IDs).
- **Sub-component:** Description only.
- **File:** read-only info.

Fields commit when focus leaves them or when you Save. **Draft with AI…** (see §12) can draft this prose for you.

---

7. Scanning your source code

Scanning parses every file in the project, applies the coding-standard profile and options, and builds the model database that all documents and exports read from. It also runs cross-file analysis (call graph, provided/required interfaces, complexity metrics, ID assignment).

Scan-page options (under the tree)

The controls beneath the tree are grouped into labelled sections:

ARXML configuration (optional) : Click **ARXML...** to attach AUTOSAR `.arxml` files. Use **Add...** (filter `*.arxml;*.xml`), **Remove**, **OK**. The status box reads *"ARXML: not configured (optional)"* or *"ARXML: N file(s) configured"*. ARXML populates SAD section 4.7 (tasks/interrupts/runnables); it is read at *generation / EA export* time, so changing it needs a re-generate, **not** a re-scan. Leave empty for non-AUTOSAR projects.

Global variable : - **Capture global variable in activity diagram** — records statements that *write* to a global (assignment, `++`/`--`, `+=`, ...) so they appear as actions in activity diagrams. *Default off.* **Scan-time** — needs a re-scan.

- **Insert blue box to indicate global variables** — adds a light-blue «global» box naming the variable with an arrow into the write action. *Default on*, but only enabled while capture is on. **Export-time** (Enterprise Architect export only) — needs only a re-export, no re-scan.

Function : - **Show parameters in function calls** — labels calls with their arguments as written in the code (e.g. `foo(&buf, len)` instead of `foo`). *Default off.* **Scan-time** — needs a re-scan.

Coding standard : The macro profile applied before parsing:

- **AUTOSAR Classic** *(default)* — rewrites AUTOSAR compiler-abstraction macros (`FUNC`, `VAR`, `CONST`, `P2VAR`, `P2CONST`, `FUNC_P2VAR`, ...) into plain C types, maps `LOCAL_INLINE`→ `static inline`, `STATIC`→`static`, `INLINE`→`inline`, and removes memory-section macros.
- **Plain C/C++** — only removes section-placement macros.

Changing the profile requires a re-scan.

Run the scan

Click the full-width **Scan** button. The project is auto-saved first (prompting for a location if it was never saved). On success the status bar summarises files/functions/interfaces and the issue count.

> A scan does **not** rebuild the SDD/SAD preview editors — they keep your edits. Use **Load from scan** in the preview toolbar to pull fresh content (which discards edits).

Custom macro rules (ExtraMacroRules)

If the parser meets a vendor macro it doesn't recognise, it records a *suspect macro* issue. In the **Issues** tab, right-click the issue → **Add ExtraMacroRule for '<macro>'...**. Choose a template:

- **Function-like macro — keep the first argument** (e.g. `FUNC(uint8, CODE)` → `uint8`)
- **Decoration macro — remove the whole call** (e.g. section/attribute macros)

The dialog pre-fills a regex **Pattern** and **Replacement** (both editable), validates the regex, and saves the rule into the project. **Save** and **re-scan** to apply. Project rules run *before* the built-in profile rules, so they can override them.

When is a re-scan required?

Change	Re-scan?
Coding-standard profile	**Yes**
Add/edit an ExtraMacroRule	**Yes**
Capture global variable toggle	**Yes**
Show parameters in function calls toggle	**Yes**
Brief-extraction settings	**Yes**
File list / tree grouping	**Yes**
Insert blue box toggle	No — re-export only
ARXML file list	No — re-generate / re-export only

---

8. Reviewing scan results

Issues tab

Lists scan warnings and errors with **Severity, Category, File, Line, Message, Code**; the tab caption shows the count. **Double-click** an entry to reveal the file in Explorer. **Right-click** for *Open file location* and, for a suspect macro, *Add ExtraMacroRule…*. Common categories include a missing file (error), a duplicate file assignment (warning), and an unrecognised macro (warning).

Scanned data tab

Browse exactly what the scan captured. Pick an entity kind, type in the **Filter** box to narrow rows, click a row for details:

- **Kinds:** Functions, Types, Variables, Files, Provided interfaces, Required interfaces, Calls, Prototypes, Macros, Includes.
- **Double-click** a row to reveal its source file; **right-click** for copy actions (cell / row / selected rows / column / all rows with headers).
- For **Functions** and **Types**, the detail pane shows a rendered activity diagram / type members, with clickable links to jump to a called function or referenced type. (Rendering needs PlantUML configured — see §17.)

Log tab

A timestamped, read-only activity log shared by every page. Watch it during scan/generate/export.

---

9. Editing the SAD / SDD (preview editors)

The **SDD Preview** and **SAD Preview** pages are split editors: Markdown source on the left, a rendered HTML + diagram preview on the right. Your edits are kept per project (saved into sidecar files) and are used verbatim at generation instead of re-rendering the template.

Preview toolbar

Button	Action
Load from scan	Rebuild this document's Markdown from the current scan — **discards your edits** (asks first).
Insert diagram ▾	Insert a ready-made PlantUML block: Sequence, State, Activity, Component, Class, or Empty.
Render preview	Render the whole document, including diagrams.
Render at cursor	Render only the ```plantuml block at the caret.
Find (Ctrl+F)	In-preview search with ▲/▼ stepping and *Match case*.
Export DOCX…	Write this document as `.md` + a rendered `.docx`.
Save…	Write the edited Markdown to a chosen `.md`.

Right-click in the editor for **Draft this section with AI…** (see §12) plus Cut/Copy/Paste/Select all. The AI item only lights up when the caret is inside a recognised section (e.g. *System Context*, *Communication*, *Purpose*, *Scope*).

Manual (free-text) sections

Content that cannot be reverse-engineered from code — system context, boot-up/shutdown description, a component's design concept, resource objectives, risks, etc. — is marked **"To be determined"** until you fill it in. Provide it either by editing the preview directly or through the AI assistant. These values are stored in the project (`ManualSections`).

---

10. Generating documents

On the **Generate** page click **Generate documents…** and choose an output folder (defaults to an `output` subfolder next to the project). Explorer opens the folder when done. A scan must have run first.

What is produced

Every generation writes, into the output folder:

File(s)	Contents
`SAD.md`	Software Architecture Design (see below)
`SDD.md`	Software Detail Design (see below)
`diagrams/**/*.puml`	PlantUML sources (interfaces, sequences, activities)

File(s)	Contents
`traceability.csv`, `traceability.md`	The traceability matrix
`model.xml`	Machine-readable model for data exchange
`metrics.csv`	Project-wide complexity metrics (and metric violations)
`issues.csv`	The full scan-issue list
`SAD.docx`, `SDD.docx`, `traceability.docx`	*Only when DOCX output is enabled*

The SAD contains: System Overview; Static View (a software-logical-architecture component diagram plus 3rd-party and in-house component tables); Interface Specification (per-component provided/required interface tables and diagrams); Dynamic View (ECU state machine, boot-up, shutdown, error handling, communication, and — from ARXML — task/ISR/runnable tables); Resource Consumption; Alternative architectures; and Appendix A, a reverse-engineering scan report (counts, parse fidelity, first 100 issues).

The SDD contains: an Introduction; then per component — description, design concept, a sub-component diagram, a file-structure table, reverse-engineered sequence diagrams (up to 10 embedded, the rest under `diagrams/sequences/`), and per software unit the type/variable tables and API definitions (prototype, parameters, return, location, calls, and a per-function activity/control-flow diagram); each component closes with an evaluation (interoperability, criticality, a complexity-metrics table, risks, testability). Function descriptions come from harvested `@brief` doc-comments.

Word (DOCX) output

DOCX is produced by a built-in Open XML writer — **no Pandoc or Word install is required**. Enable it globally with **Options ▸ Settings ▸ "Also export DOCX (SAD/SDD/traceability) when generating documents"**, or add `--docx` on the CLI. You can also export a single previewed document with the **Export DOCX…** toolbar button.

To embed diagrams as **images** in the DOCX (rather than monospaced text), configure PlantUML — see §17. To apply your organisation's Word styles, set **Word style reference (.docx)** in Settings.

`@brief` doc-comment extraction

During a scan RE4A harvests the one-line *brief* from the doc-comment above each function/type (it understands Doxygen `/**` and `/*!`, Vector banner blocks, one-line `/** \brief … */`, and the AUTOSAR backslash form). Briefs become the function/type description in the SDD and the note on Enterprise Architect elements. It is on by default; tune it via the project's brief-extraction settings (tags, max length, first-line fallback). Changing these needs a re-scan.

---

11. Templates

Document layout is driven by **Scriban** templates `SAD.sbn` and `SDD.sbn`. RE4A ships built-in defaults and lets you override them with your own **template sets**.

Use a template set

On the **Generate** page use the **Template:** dropdown to pick **Built-in (default)** or a named set; click ↻ to refresh the list. A template set is simply a subfolder under the *Templates folder* (default `%APPDATA%\RE4A\templates`, changeable in **Options ▸ Settings**) that contains `SAD.sbn` and/or `SDD.sbn` (a missing file falls back to the built-in one). On the CLI, point `gen`/`all` at a folder with `--templates <dir>`.

AI Template Builder

Turn an existing example document into a matching template:

1. **Generate ▸ AI Template Builder...**
2. Pick **SAD** or **SDD**, **Choose example...** (`.docx` or `.md`).
3. **Generate template with AI** — the result is auto-validated by actually rendering it (with one automatic repair round). The status turns green when it renders cleanly, red if it still errors.
4. Review/edit the `.sbn`, then **Save as template set...** and name it. It appears in the Template dropdown.

The builder only uses real data keys (it never invents fields) and writes "*To be determined*" where your layout needs data RE4A does not have. It requires **AI assist enabled** (see §12).

Brief template (AI)

Infer which doc-comment tags mark the *brief* in **your** codebase:

1. **Generate ▸ Brief template (AI)... → Choose source files...**.
2. Review the extracted comment blocks, tick the consent checkbox "*I consent to send the comment blocks above to <provider>*".
3. **Generate template with AI**, review/edit the JSON, **Save into project**. **Re-scan** to harvest briefs.

Only comment text is sent (never source code), and only after the explicit consent tick.

---

12. AI assist (optional)

RE4A includes an optional assistant that can **draft the free-text prose** sections and build templates. It is **off by default**, and nothing is ever sent until you enable it and explicitly click a Draft button.

> **Privacy:** only *derived metadata* is sent — component/interface/function names, signatures, structural counts and complexity metrics. **Source code is never sent** (the scan database does not even store function bodies). You can preview the exact payload before it goes out. The one exception is the *Brief template* builder, which sends selected files' doc-comment blocks after a separate consent tick.

Enable and configure

Options ▸ AI assist...:

- Tick **Enable AI assist**.
- **Provider:** Anthropic, OpenAI, Azure OpenAI, or **Ollama (local)**.
- **Endpoint:** blank = provider default (Azure requires your resource URL, e.g. `https://my-res.openai.azure.com`).
- **Model / deployment:** type it, or click **Load** to fetch the provider's list (examples: `claude-sonnet-5`, `gpt-4o`, `llama3.1`).
- **API key:** paste it (Ollama needs none). Use **Test connection** to verify.

- **Max tokens:** default 1024 (range 64–8192).

API keys are stored **encrypted for your Windows user only** (DPAPI) at `%LOCALAPPDATA%\RE4A\ai-credentials.dat` — never in a project or settings file. For CI you can instead set the `RE4A_AI_KEY` environment variable (which overrides the stored key). Non-secret settings live in `%APPDATA%\RE4A\ai-settings.json` and are shared by all projects.

Draft prose

- **Component / sub-component:** select it, click **Draft with AI…** in the Properties tab, pick the section (Description / Goal / Design concept), review the payload, **Generate**, edit, then **Accept**.
- **Document sections:** in the SAD/SDD preview, place the caret in a supported section and use the **Draft this section with AI…** context-menu item.

Metadata-backed sections (SAD System Context, Communication; SDD Purpose, Scope) get finished prose; design-intent sections (state machine, boot-up, shutdown, error handling, etc.) get a fill-in scaffold with bracketed placeholders rather than invented facts. A scan must have run first.

---

13. Enterprise Architect export

Exports the model as a **Sparx Enterprise Architect project (`.qea`)** whose package tree mirrors the SAD/SDD outline, with **pre-drawn** component, interface and per-function activity diagrams (real EA shapes — no images, and no PlantUML/Java/Graphviz needed for these).

Requirements

- **Enterprise Architect 16 or 17 (64-bit)** to open the `.qea`.
- A **blank** `EABase.qea` template from your own EA installation. RE4A does not bundle it (Sparx licence). By default RE4A auto-discovers it under `Program Files\Sparx Systems\*\EABase.qea`; you can override the path in **Options ▸ Settings ▸ "EA base project (EABase.qea)"** or with `--base`. The base must be an untouched, blank EA project.
- A completed scan.

Run it

Generate ▸ Export to Enterprise Architect (.qea)… → pick a level → choose the output `.qea`. The file is revealed in Explorer when done.

What you get

- Two top-level packages, **SAD** and **SDD**, mirroring the documents; sections a human must still write become *Note* placeholders ("*To be determined…*").
- A **Software Logical Architecture** component diagram, per-component **interface** diagrams, and per-component **sub-component** diagrams.
- One **activity diagram per function**, reconstructed from its control flow (branches, loops, merges). Optionally annotated with global-variable writes and call parameters (see §7).
- **Call navigation:** a call to another exported function is a *CallBehavior* action — double-click it in EA to jump to the callee's diagram (indirect calls are labelled "*(indirect)*").

- **Traceability connectors:** provided interfaces realised from their component, required interfaces used from the provider, component/unit dependencies, and each SAD interface traced to its implementing function («trace»). If ARXML is configured, OS runnables are traced to their C function.

Export levels

Four cumulative levels (each includes the previous):

Level	Adds
Architecture	Components per core + the logical-architecture diagram
Architecture + interfaces	SAD interfaces with realise/use connectors + one interface diagram per component
+ Interface functions	The SDD branch: sub-components + public-API functions as navigable activities + interface→function traces
+ Detailed design (SDD) *(default, recommended)*	Internal (static/private) functions, types, global variables, file artifacts, sub-component interfaces

> Element GUIDs are **deterministic** across re-exports, so you can compare/merge models in EA. **Macros are never exported to EA.** Sub-components appear only from *+ Interface functions* upward. Very large functions have their activity diagram truncated (a "*… (flow truncated)*" action) at about 260 shapes.

---

14. ALM export (ReqIF and REST push)

Get the scanned architecture (components, interfaces and their trace links) into a requirements/ALM tool. Both paths are **one-way** and **idempotent**: repeated exports keep stable identifiers, so re-imports/re-runs **update** the same items instead of duplicating them, and **nothing is ever deleted** on the ALM side. Both require a completed scan and live on the **Generate** page.

> **Identifier stability** depends on a per-project *ALM namespace GUID* that RE4A assigns and saves into the `.re4a.json` on the first export. Keep it stable — if it changes, a re-import duplicates everything.

Export levels (both paths)

Level	Contents
Architecture	Components + sub-components joined by *contains* links
Architecture + interfaces *(recommended default)*	Adds component-level interfaces with provides/requires links
+ Interface functions	Adds the public-API functions that implement each interface
+ Detailed design (SDD)	Adds sub-component interfaces, internal functions, types, globals and **macros** — can be large

ReqIF file export

Generate ▸ Export to ALM (ReqIF)... → pick a level → optionally tick **Include activity diagrams for functions** → **Export...** → choose the output file. Produces an OMG ReqIF 1.2 file (`.reqifz`, or plain `.reqif` XML when diagrams are off) that Polarion, codeBeamer and DOORS/DOORS Next import natively. Embedding diagrams needs PlantUML configured and forces a `.reqifz` archive.

Direct REST push (Polarion / codeBeamer)

Generate ▸ Push to ALM (REST)... writes straight into the server — no file, no manual import.

1. **Platform:** Polarion or codeBeamer.
2. **Server URL:** must be `https://` (plain HTTP is refused; loopback is allowed for local test servers).
3. **Project ID** (Polarion) / **Tracker ID** (codeBeamer).
4. **External-id field:** the custom field that stores RE4A's idempotency key (default `re4aExternalId`). It must already exist on the target.
5. **Credential:** paste the **Access token (PAT)** (Polarion) or **Credential (user:token)** (codeBeamer). It is stored encrypted for your Windows user only (DPAPI) — never in the project. Leave blank to reuse a previously stored one.
6. Keep **Dry run** ticked (default) and click **Push...** to preview the exact create/update/link plan without writing anything. Use **Test connection** to validate first.
7. Untick **Dry run** and push for real when the plan looks right.

> **Polarion** needs its REST API enabled on the server (off by default) and a Personal Access Token. **codeBeamer** creates items one-by-one and self-throttles (~3 requests/s); association type names used for links must already exist on the server. Items you removed from the source are **left untouched** on the ALM side (they are never deleted). Only a *real* push saves the (non-secret) target settings back into the project — a dry run never rewrites it.

For CI, supply the token via the `RE4A_ALM_TOKEN` environment variable (it takes precedence and is never persisted).

---

15. Settings reference

Options ▸ Settings (global, all projects)

Stored in `%APPDATA%\RE4A\settings.json`. Each path field has a **Browse...**; leave a field empty to auto-discover.

Field	Purpose
PlantUML (.jar or .exe)	Renderer for diagram images. Empty = diagrams stay as text.
Java runtime (java.exe)	Needed to run `plantuml.jar`. Empty = `java` from PATH / `JAVA_HOME`.
Graphviz layout (dot.exe)	Better layout for large diagrams. Empty = built-in *smetana*.
Word style reference (.docx)	Imports Word styles into generated `.docx`.
Templates folder	Root for template sets. Empty = `%APPDATA%\RE4A\templates`.
EA base project (EABase.qea)	Blank EA template for the `.qea` export. Empty = auto-discover.

Field	Purpose
Also export DOCX… *(checkbox)*	Also write SAD/SDD/traceability as `.docx` on Generate.
Test rendering *(button)*	Verify the PlantUML/Java/Graphviz toolchain.

Auto-discovery searches a **`tools`** folder next to the project, its parent, and the app for `plantuml*.jar` / a JRE / `dot.exe`, and probes `PATH`, `JAVA_HOME` and `GRAPHVIZ_DOT`.

> Tool paths, the DOCX flag, the templates folder and AI settings are **global** (per user/machine), **not** stored in the project. Older tool-path keys in a `.re4a.json` are ignored.

---

16. Command-line interface

`re4a` (`RE4A.Cli.exe`) drives the whole pipeline from a terminal — useful for automation and CI. It uses the **same license** and project files as the desktop app. Run `re4a` with no arguments to print usage.

> The examples below write `re4a` for brevity. Invoke the shipped executable as **`RE4A.Cli.exe <verb>`** (or rename it to `re4a.exe`, or run `dotnet RE4A.Cli.dll <verb>` from source).

> **Exit codes:** `0` = success · `1` = usage / runtime error · `3` = licensing failure. Every verb except help requires a valid installed license.

Verb	Purpose
`suggest`	Propose a project config from a source folder
`scan`	Parse the code into the model database
`gen`	Generate SAD/SDD, diagrams, traceability, metrics
`all`	`suggest` + `scan` + `gen` in one step
`reqif`	Export an OMG ReqIF package for ALM import
`ea`	Export an Enterprise Architect `.qea` project
`push`	Push the model to Polarion / codeBeamer over REST

```
# Propose a project from a source folder (writes <folder>.re4a.json)
re4a suggest C:\src\MyEcu [MyEcu.re4a.json]

# Build the model database (writes MyEcu.db next to the config)
re4a scan MyEcu.re4a.json

# Generate documents (auto-scans first if the .db is missing)
re4a gen MyEcu.re4a.json -o C:\out --docx [--templates <dir>]

# suggest + scan + gen in one step
re4a all C:\src\MyEcu -o C:\out --docx

# Export ReqIF for ALM
re4a reqif MyEcu.re4a.json -o MyEcu.reqifz --level detailed --diagrams
```

```
# Export an Enterprise Architect project
re4a ea MyEcu.re4a.json -o MyEcu.qea --base EABase.qea --level detailed

# Push to Polarion / codeBeamer (dry-run first!)
re4a push polarion   MyEcu.re4a.json --server https://alm.example.com --project
MYPROJ --level interfaces --dry-run
re4a push codebeamer MyEcu.re4a.json --server https://cb.example.com  --tracker
1234    --level interfaces --dry-run
```

Notes and defaults

- The **scan database** is always the config path with a `.db` extension (`MyEcu.re4a.json` → `MyEcu.db`). `reqif`, `ea` and `push` require it to exist; `gen`/`all` auto-scan if it is missing.
- `--level` accepts `architecture | interfaces | functions | detailed` (aliases: `full` = functions, `sdd` = detailed). Default is **interfaces** for `reqif`/`push`, but **detailed** for `ea`.
- **Default output:** `gen` writes to `{configFolder}\output`; `all` writes to `{currentDir}\output`; `reqif`/`ea` default their file next to the config (`.reqifz` / `.qea`).
- `--diagrams` (ReqIF) and embedded PNG diagrams (`gen`) need PlantUML configured; `--diagrams` forces a `.reqifz`.
- `push` cannot enter a credential interactively — store one via the GUI push dialog first, or set `RE4A_ALM_TOKEN`. It also needs the container id (`--project` for Polarion, `--tracker` for codeBeamer) and a server URL.
- The first `reqif` / `ea` export writes the project's ALM namespace GUID back into the `.re4a.json`.

---

17. Diagram rendering tools

RE4A always writes diagrams as **PlantUML text**. To turn them into **images** (for the DOCX output, the ReqIF `--diagrams` option, and the in-app previews / Scanned-data activity view), install a renderer:

1. Download **PlantUML** — the `plantuml-mit` build of `plantuml.jar` is recommended.
2. Install a **Java** runtime (a portable JRE is fine).
3. *(Optional)* Install **Graphviz** for higher-quality layout of large diagrams.
4. In **Options ▸ Settings**, set **PlantUML (.jar or .exe)**, **Java runtime (java.exe)** and optionally **Graphviz layout (dot.exe)**. Click **Test rendering** to confirm.

Alternatively, drop `plantuml*.jar` (and a JRE / `dot.exe`) into a `tools` folder next to your project — RE4A finds it automatically.

Without a renderer, diagrams simply appear as monospaced PlantUML text; nothing else fails. (The native Enterprise Architect activity diagrams do **not** need any of this — they are drawn directly.)

> When rendering `.puml` files **outside** RE4A, set `PLANTUML_LIMIT_SIZE=16384` — large components exceed PlantUML's default 4096-px canvas and get clipped otherwise. RE4A sets this automatically when it renders for DOCX.

---

18. Troubleshooting

Symptom	Cause / fix
App is locked, red banner	No usable license. Click **License…**, import a valid `.lic`. See §3.
CLI exits with code 3	Licensing failure. Activate in the desktop app first; the CLI reuses that license.
"Run a scan first"	Generate / export / AI needs the scan database. Click **Scan** (or `re4a scan`).
My option change had no effect	It may be a *scan-time* option — re-scan. See the re-scan table.
Preview edits vanished after re-scan — they didn't	A re-scan keeps your preview edits. Use **Load from scan** only if you *want* to overwrite them.
Diagrams show as text, not images	No PlantUML/Java configured, or it failed. See §17 and check the **Log** tab.
Large diagram is clipped when rendered externally	Set `PLANTUML_LIMIT_SIZE=16384`.
EA export fails: no base project	Install EA 16/17, or set **Options ▸ Settings ▸ EA base project** / `--base` to a blank `EABase.qea`.
A file shows as `FileNotFound` / `Partial`	Missing file → fix the path and re-scan. `Partial` means the parser skipped a region (e.g. `#if/#else` mid-statement) — the rest of the file is still captured.
Suspect / unexpanded macro warnings	Add an **ExtraMacroRule** from the Issues tab, Save, and re-scan.
ALM re-import duplicated everything	The project's ALM namespace GUID changed — do not hand-edit or lose it.
ALM push refuses the server URL	Use `https://` (or loopback). Plain HTTP is rejected.
"Clock tampered" lock	System clock was set earlier than a previous run. Fix the date/time; if it persists, re-import the `.lic`.

---

19. Reference appendix

Project file (`*.re4a.json`)

A plain JSON file you may also hand-edit. Only non-default values are written on save, and unknown keys from newer versions are preserved.

Key	Meaning	Default
`Name` / `EcuName`	Project / ECU name	source-folder name
`SourceRoot`	Source root (absolute, or relative to the config folder)	—

Key	Meaning	Default
`CodingProfile`	Coding-standard profile name	`AUTOSAR Classic`
`ExtraMacroRules[]`	`{ Pattern, Replacement }` regex rewrites (run before the profile)	empty
`Cores[] › Components[] › SubComponents[] › Files[]`	The grouping hierarchy; file paths use `/`	—
`Component.Id`	SAD table id	auto `SWC_001…`
`Component.Description` / `Goal`	Free text	empty
`Component.IsThirdParty` / `ThirdPartyName`	Bought-in marker	false
`Component.IsSafety`	Safety component (`Safe` id prefix)	false
`ManualSections{}`	Free-text sections, keyed (e.g. `sad.system_context`)	empty → "*To be determined*"
`ArxmlFiles[]`	AUTOSAR files for SAD 4.7	empty
`BriefExtraction`	`Enabled`, `Tags`, `FallbackToFirstLine`, `MaxLength`	on, `@brief`/`\brief`, off, 300
`CaptureGlobalWriteActions`	Capture global writes (scan-time)	false
`InsertGlobalWriteBox`	Blue «global» box (EA export-time)	true
`ShowCallArguments`	Show call arguments (scan-time)	false
`AlmNamespace`	Stable identity for ALM/EA exports	assigned on first export
`AlmTarget`	Saved ALM push target (non-secret)	—

Output files (from *Generate*)

`SAD.md`, `SDD.md`, `diagrams/**/*.puml`, `traceability.csv`, `traceability.md`, `model.xml`, `metrics.csv`, `issues.csv`, and (with DOCX enabled) `SAD.docx`, `SDD.docx`, `traceability.docx`.

On-disk storage locations

Path	Contents
`<project>.re4a.json`	The project
`<project>.db`	The scan database (next to the project)
`%APPDATA%\RE4A\settings.json`	Global tool paths, templates folder, EA base, DOCX flag
`%APPDATA%\RE4A\ai-settings.json`	AI provider settings (non-secret)
`%APPDATA%\RE4A\templates\`	Template sets

Path	Contents
`%LOCALAPPDATA%\RE4A\license.lic`	Installed license
`%LOCALAPPDATA%\RE4A\ai-credentials.dat`	AI API key (DPAPI-encrypted)
`%LOCALAPPDATA%\RE4A\alm-credentials.dat`	ALM tokens (DPAPI-encrypted)

Keyboard shortcuts

Shortcut	Action
Ctrl+N	New project from folder
Ctrl+O	Open project
Ctrl+S	Save project
Ctrl+Shift+S	Save project as
Ctrl+F	Find (in a preview page)
Ctrl / Shift + click	Multi-select tree nodes of the same kind

Environment variables (for CI / automation)

Variable	Purpose
`RE4A_ALM_TOKEN`	ALM access token (overrides the stored one; never persisted)
`RE4A_AI_KEY`	AI API key (overrides the stored one)

RE4A — Reverse Engineering for Automotive. This guide reflects the application's current feature set. Screens and labels quoted here match the shipping build.